



Comment fonctionne un émulateur Super Nintendo

Une explication pédagogique, du processeur au pixel et au son

Émulateur écrit **from scratch en Rust** — CPU 65C816, PPU graphique, APU sonore (SPC700 + S-DSP), DMA/HDMA, cartouches LoROM/HiROM.

Document technique et pédagogique · Version 1.0

1. Qu'est-ce qu'émuler une console ?

Une console de jeu comme la Super Nintendo (SNES / Super Famicom, 1990) est un ensemble de **puces électroniques** qui coopèrent : un processeur, une puce vidéo, une puce audio, de la mémoire. **Émuler** la console, c'est écrire un programme qui **imite le comportement de chacune de ces puces**, cycle par cycle, de façon assez fidèle pour que le code d'origine d'un jeu — les mêmes octets que sur la cartouche — s'exécute et produise exactement la même image et le même son.

💡 Analogie

Émuler, ce n'est pas « refaire le jeu ». C'est construire une **fausse console** si convaincante que la vraie cartouche ne voit pas la différence. Le jeu croit parler à une vraie puce vidéo ; en réalité il parle à quelques milliers de lignes de Rust.

La SNES est une machine **16 bits**. Ses composants principaux sont :

- **Le CPU 65C816** — le cerveau : il exécute la logique du jeu.
- **Le PPU** (*Picture Processing Unit*) — la puce graphique : elle dessine l'écran.
- **L'APU** (*Audio Processing Unit*) — un mini-ordinateur sonore autonome.
- **La mémoire** — WRAM (128 Ko de travail), VRAM (vidéo), et la cartouche (ROM + sauvegarde).
- **Le DMA** — un déménageur d'octets ultra-rapide entre ces mémoires.

2. Vue d'ensemble : qui parle à qui

Tout est relié par un **bus** : une autoroute d'adresses par laquelle le CPU lit et écrit. Quand le CPU écrit à une certaine adresse, ce n'est pas toujours de la mémoire — cela peut être un **registre** qui commande le PPU, l'audio ou un transfert. C'est le rôle du bus de **router** chaque accès au bon endroit.



🔑 À retenir

Dans notre code, cette structure se retrouve telle quelle : un module `cpu`, un module `ppu`, un module `apu`, un `bus` qui les relie, et un `scheduler` (l'horloge) qui garde tout le monde synchronisé. Le cœur d'émulation (crate `core`) n'a **aucune dépendance d'entrée/sortie** : il est testable tout seul. L'affichage et le clavier vivent dans un binaire séparé.

3. La cartouche et la carte mémoire

Le 65C816 adresse la mémoire sur **24 bits** : 16 Mo d'espace, découpé en 256 « banques » (*banks*) de 64 Ko. Mais toutes les adresses ne mènent pas à de la RAM. Selon la zone, un même accès touche la

mémoire de travail, une puce, ou la cartouche :

\$00–\$3F : \$0000	Miroir des 8 premiers Ko de la WRAM
\$2100–\$21FF	Registres du PPU (vidéo)
\$2140–\$2143	Ports de l'APU (son)
\$4200–\$437F	Registres CPU internes + DMA
\$8000–\$FFFF	ROM de la cartouche
\$70–\$7D : \$0000	Sauvegarde SRAM (pile de la cartouche)
\$7E–\$7F : entier	128 Ko de WRAM (mémoire de travail)

LoROM et HiROM

Les cartouches câblent leur ROM de deux façons principales : **LoROM** (blocs de 32 Ko en haut de chaque banque) et **HiROM** (banques pleines de 64 Ko). L'émulateur lit l'en-tête interne de la ROM pour deviner le bon schéma — il attribue un **score** à chaque hypothèse (le couple somme de contrôle / complément doit valoir \$FFFF, le titre doit être lisible...) et garde le meilleur. Il en déduit aussi la **région** (PAL/NTSC) et la taille de la sauvegarde.

💡 Analogie

L'*open bus* est un détail subtil mais crucial. Sur le vrai matériel, lire une adresse « qui ne mène nulle part » ne renvoie pas 0, mais **la dernière valeur qui a circulé sur le bus** (une sorte d'écho électrique). Certains jeux en dépendent : il faut donc mémoriser cette dernière valeur (le registre **MDR**) et la renvoyer pour les adresses non mappées.

4. Le cœur : le processeur 65C816

Le CPU exécute la logique du jeu : « si le joueur appuie à droite, avancer Mario ». Il ne fait rien de magique — il répète sans cesse un cycle de trois temps :

1· Lire l'instruction (fetch) → 2· La décoder → 3· L'exécuter ↻

Les registres

Le 65C816 possède une poignée de **registres** — de minuscules cases de mémoire internes, ultra-rapides : l'accumulateur **A** (pour les calculs), deux index **X** et **Y**, un pointeur de pile **S**, le compteur de programme **PC** (l'adresse de la prochaine instruction), et un registre d'état **P** qui contient des **drapeaux** (résultat nul ? négatif ? retenue ?).

La particularité 8/16 bits

Le 65C816 est un descendant du 6502 (8 bits) de la NES. Il démarre en **mode émulation** (il se fait passer pour un 6502) puis bascule en **mode natif** 16 bits. Encore mieux : deux drapeaux (**M** et **X**) permettent au jeu de choisir **à la volée** si l'accumulateur et les index travaillent sur 8 ou 16 bits. Une

même instruction peut donc se comporter différemment selon le contexte — l'émulateur teste ces drapeaux à chaque opération.

```
// Le squelette d'un pas de CPU (simplifié)
fn step(cpu, bus) {
    if une_interruption_est_en_attente() {
        servir_l_interruption(); // NMI (vsync) ou IRQ (timer)
        return;
    }
    let opcode = bus.read(cpu.pc); // 1. fetch
    cpu.pc += 1;
    match opcode {
        // 2. décode (256 cas)
        0xA9 => lda_immediat(cpu, bus), // 3. exécute : charger A
        0x8D => sta_absolu(cpu, bus),   // ranger A en mémoire
        // ... les 254 autres opcodes ...
    }
}
```

Notre implémentation couvre les **256 instructions**, tous les modes d'adressage (comment calculer l'adresse d'un opérande), l'arithmétique décimale (BCD, pour les scores), et les instructions spéciales (`WAI` qui met le CPU en veille jusqu'à la prochaine interruption, les déplacements de blocs mémoire...).

🔑 À retenir

Le temps se compte en cycles. Chaque lecture ou écriture en mémoire fait avancer l'« horloge maître » d'un nombre précis de cycles, qui dépend de la zone visée (la ROM rapide, la RAM, les manettes... n'ont pas la même vitesse). C'est ainsi que l'émulateur sait *combien de temps* a passé, et donc quand déclencher les événements vidéo et audio.

5. Le chef d'orchestre : le timing

La SNES bat au rythme d'une horloge maître d'environ **21,5 MHz**. L'image se construit ligne par ligne : **262 lignes** en NTSC (60 Hz), **312 lignes** en PAL (50 Hz — le cas de vos jeux européens). Chaque ligne dure 1364 cycles.

À la fin de la zone visible (ligne 225) commence le **V-blank** : le canon à électrons « remonte » en haut de l'écran. C'est le seul moment sûr pour modifier la vidéo. La console prévient le CPU par une **interruption** :

- **NMI** (*Non-Maskable Interrupt*) : déclenchée à chaque V-blank. C'est le battement de cœur du jeu — il y met à jour l'affichage, lit les manettes, avance la musique.
- **IRQ** : déclenchée à une ligne/position choisie par le jeu. Elle sert aux effets « à mi-écran » (barre de statut fixe pendant que le décor défile, par exemple).

🔑 À retenir

Le `scheduler` tient une seule horloge (un compteur 64 bits) et un « prochain événement ». À chaque accès du CPU, il compare l'horloge à cet horizon ; quand il est franchi, il traite la fin de ligne : dessiner la scanline, déclencher NMI/IRQ, rattraper l'audio. Une seule comparaison par accès — c'est ce qui rend l'émulation rapide.

6. La puce graphique : le PPU

Le PPU est le composant le plus riche — et le plus délicat à émuler fidèlement. Son rôle : transformer des **tuiles** et des **sprites** rangés en mémoire vidéo en une image de 256×224 pixels.

Trois mémoires vidéo

- **VRAM** (64 Ko) : les *tuiles* (petits carrés de 8×8 pixels) et les *plans de tuiles* (quelles tuiles à quel endroit).
- **CGRAM** (256 couleurs) : la palette. Chaque pixel ne stocke pas une couleur mais un *numéro* de couleur dans cette palette.
- **OAM** : la table des sprites (position, tuile, priorité de chacun des 128 objets mobiles).

💡 Analogie

Pensez à un tableau de feutrine avec des gommettes. Les **tuiles** sont un stock de petits motifs réutilisables ; le **plan de tuiles** dit où coller chaque motif ; la **palette** dit quelles couleurs donner aux motifs. Redessiner l'écran, c'est relire cette recette 60 fois par seconde.

Les couches de fond et les modes

Le PPU superpose jusqu'à 4 **couches de fond** (BG1–BG4). Un registre choisit le **mode** (0 à 7), qui décide combien de couches existent et avec combien de couleurs. Le fameux **Mode 7** est à part : il ne fait qu'une seule couche, mais qu'il peut **faire pivoter et mettre à l'échelle** par une transformation mathématique — d'où les circuits de F-Zero ou la carte du monde qui tournent.

Le rendu ligne par ligne

Notre PPU dessine **une scanline à la fois**, à la fin de chaque ligne visible. C'est essentiel : comme le jeu peut changer les registres vidéo *entre* deux lignes, dessiner ligne par ligne reproduit naturellement presque tous les effets d'écran. Pour chaque ligne, le pipeline est :



Les **priorités** décident qui passe devant qui (Mario devant le décor mais derrière un nuage). Les **fenêtres** masquent des zones (le rond de lumière d'une torche dans une pièce sombre). La **color math** additionne ou soustrait deux « écrans » pour les transparences, ombres et fondus — c'est elle qui, par exemple, fait apparaître le menu de sélection de *Super Mario All-Stars* (superposition de deux couches).

7. Le déménageur : DMA et HDMA

Copier des milliers d'octets vers la VRAM un par un avec le CPU serait trop lent. Le **DMA** (*Direct Memory Access*) le fait à la place, en bloc, à ~1 octet tous les 8 cycles. Le jeu remplit quelques registres (source, destination, taille) puis lance le transfert ; le CPU est mis en pause le temps de la copie.

Le **HDMA** est plus malin : il transfère **un petit paquet à chaque ligne d'écran**, automatiquement, juste avant que la ligne soit dessinée. C'est ce qui permet les **dégradés de ciel** (une couleur différente par ligne) ou la perspective du Mode 7 (une échelle différente par ligne) sans effort pour le CPU.

🔑 À retenir

Détail qui fait tout : le HDMA d'une ligne doit s'exécuter **avant** le rendu de cette ligne. Se tromper d'une ligne, et le dégradé est décalé. C'est l'une des sources de bugs les plus classiques — d'où l'attention portée à l'ordre des événements dans le scheduler.

8. Le son : l'APU (SPC700 + S-DSP)

Surprise : la partie audio de la SNES est **un ordinateur à part entière**, avec son propre processeur (le **SPC700**), sa propre mémoire (64 Ko) et sa puce de synthèse (le **S-DSP**). Il ne partage rien avec le CPU principal, sauf **quatre boîtes aux lettres** (les ports \$2140–\$2143).

💡 Analogie

Le CPU principal et l'APU sont comme deux bureaux qui ne communiquent que par quatre cases postales. Au démarrage, un petit programme gravé dans l'APU (l'*IPL ROM*, 64 octets) répond « je suis prêt » (\$AA, \$BB) ; le jeu lui envoie alors, case par case, tout son **moteur sonore** et ses instruments. La quasi-totalité des jeux *attendent* cette poignée de main avant de continuer — l'émuler est donc indispensable, même sans le son.

Comment naît un son

Le S-DSP mélange **8 voix** en un flux stéréo à 32 kHz. Pour chaque voix :

- **BRR** : les instruments sont des échantillons compressés (blocs de 9 octets). On les décompresse à la volée.
- **Hauteur (pitch)** : on lit l'échantillon plus ou moins vite pour jouer une note grave ou aiguë, avec une interpolation gaussienne pour adoucir.
- **Enveloppe ADSR** : le volume évolue dans le temps (attaque, chute, maintien, extinction) — c'est ce qui distingue un piano d'un orgue.
- **Écho** : un tampon circulaire en mémoire recrée la réverbération si caractéristique des musiques de la SNES (les grottes de *Secret of Mana*).

L'émulateur fait tourner le SPC700 « en rattrapage » : le bus le fait avancer juste avant chaque accès aux boîtes aux lettres et une fois par ligne, avec une conversion d'horloge en virgule fixe pour ne jamais dériver. Les échantillons produits sont ensuite ré-échantillonnés vers la carte son de l'ordinateur,

avec un léger ajustement dynamique du débit pour absorber la différence entre l'horloge émulée et celle du PC (une dérive devient un imperceptible vibrato plutôt qu'un craquement).

9. La boucle : comment tout se synchronise

Assembler tout cela tient en une boucle. Pour produire une image :

```
fn run_frame(inputs) {
    // Tant que la trame n'est pas finie (fin de l'écran) :
    while !fin_de_trame {
        cpu.step(bus);          // le CPU exécute une instruction...
        // ...chaque accès mémoire a fait avancer l'horloge, ce qui
        // déclenche au bon moment : rendu d'une scanline, NMI/IRQ,
        // HDMA, rattrapage de l'audio.
    }
    return framebuffer; // l'image de 256×224 est prête
}
```

Le binaire d'interface appelle `run_frame` à la cadence de la région (50 ou 60 fois par seconde), convertit l'image en pixels à l'écran (fenêtre `winit` + `pixels`), envoie le son à la carte audio (`cpal`) et transmet l'état du clavier comme s'il s'agissait d'une manette.

10. Au-delà de la console

Parce que l'émulateur **possède la totalité de l'état** de la machine, il peut faire des choses impossibles sur le vrai matériel :

- **Sauvegardes SRAM** : la pile de sauvegarde de la cartouche est un simple fichier `.srm` à côté de la ROM, chargé au lancement, réécrit en quittant.
- **Save states** : on sérialise *tout* l'état (CPU, PPU, APU, mémoires) dans un fichier. Le recharger fait reprendre le jeu exactement où il en était — au pixel et à l'échantillon près (c'est vérifié : après sauvegarde/rechargement, l'émulation reste identique bit pour bit).
- **Débogage** : traces d'instructions, captures d'écran, inspection de la mémoire — les outils qui ont permis de construire et de valider l'émulateur.

11. En résumé

Émuler la SNES, c'est **faire cohabiter fidèlement une demi-douzaine de puces** : un CPU qui exécute la logique, une horloge qui rythme tout, un PPU qui compose l'image ligne par ligne, un DMA qui déplace les données à temps, et un second ordinateur dédié au son. La difficulté n'est pas une puce en particulier — c'est leur **synchronisation exacte**, car les jeux exploitent le moindre détail de timing. C'est pourquoi chaque composant est écrit d'après une référence matérielle vérifiée, puis validé sur de vrais jeux commerciaux.

 À retenir

Le principe directeur : ne jamais inventer une constante matérielle. Chaque coût en cycles, chaque bit de registre, chaque table de priorités provient d'une documentation de référence — et est vérifié de façon adversariale. C'est ce qui fait la différence entre « ça marche à peu près » et « le jeu s'exécute vraiment comme sur la console ».

Document généré pour accompagner l'émulateur SNES écrit en Rust · $\approx 16\,000$ lignes, 219 tests · CPU 65C816, PPU (modes 0–7, color math, HDMA), APU (SPC700 + S-DSP), save states.